```
    month: 13,
    day: 32,
    year: 2021
));
```

```
Output:
bool(true)
bool(false)
```

date_default_timezone_get(), date_default_timezone_set()

We've seen this function already in the previous section, when comparing `DateTime` objects built with timestamps vs. formatted strings. The `get` form will report the configured (via the `date.timezone` .ini directive) default time zone. The `set` form, when called with a valid string containing a canonical time zone name, will change it. (There is some support for aliases, but it is recommended that valid canonical `tzdata` names are always used.)

All `DateTime` objects created with formatted date strings and without time zone data will have their time zone set to default:

```
date_default_timezone_set('America/Los_Angeles');
$dt = new DateTime('2024-09-01');
print $dt->format('Y-m-d H:i:s T') . PHP_EOL;

date_default_timezone_set('Europe/London');
$dt = new DateTime('2024-09-01');
print $dt->format('Y-m-d H:i:s T');
```

```
Output:
2024-09-01 00:00:00 PDT
2024-09-01 00:00:00 BST
```

date_parse(), date_parse_from_format()

These two functions take a date-formatted string (and in the case of `date_parse_from_format()`, another string defining the format) and return an array of useful information such as the date and time components (year, day, hour, etc.), time zone information if present, and even warnings and errors if there was a problem with the input:

```
print_r(date_parse('2024-09-01'));
```

```
Output:
```